Chapter route

How can a filesystem maintain accessible, recoverable persistent objects using finite blocks, metadata, and recovery protocols?

Prerequisites are the file interface, logical/physical blocks, and basic journaling.

- Trace inodes, directories, open tables, and page caches.
- Compare contiguous, linked, and indexed allocation.
- Analyze metadata/data consistency across failures.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Logical files and recoverable disk structures

Use a teaching volume with 4 KiB blocks:

Block	Persistent structure	Contents
0	Superblock	Block size/count, inode/free-space locations
1	Inode bitmap	Allocated inodes
2	Data-block bitmap	Free data blocks
3	Inode table	Type, size, mode, direct/indirect pointers
4	Root directory	home→inode 3
5	/home directory	u→inode 5
6	/home/u directory	notes.txt→inode 7
8–10	File data	Three 4 KiB blocks of notes.txt

The basic filesystem translates logical blocks into device blocks; file organization manages free space and logical-to-physical mapping; the logical filesystem manages directories, FCBs/inodes, and APIs; I/O control and drivers perform transfers. Caches may reside above these layers, but the system contract must define the durable commit point.

An append must preserve

$$\text{name} \to \text{valid inode} \to \text{consistent block set}, \qquad \text{allocated} \cap \text{free} = \varnothing.$$

Logical files and recoverable disk structures (continued)

A normal write return does not make the directory entry, inode, bitmap, and data simultaneously durable. Unmount, successful flush, and transaction completion are different events.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

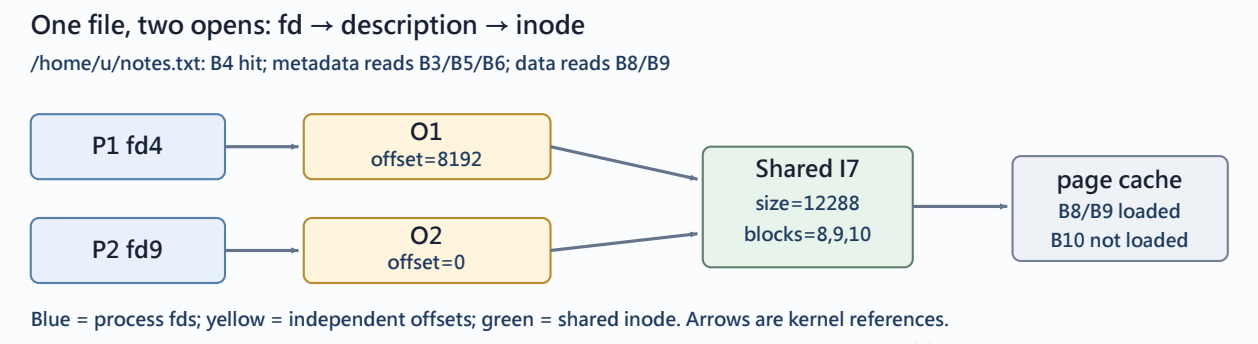
Complete path walk, open, and read states

Use `/home/u/notes.txt`, initially caching only root-directory block 4:

Step	Lookup	Read or hit	Result
1	home in /	Root block 4 hit	Inode 3
2	Inode 3	Cold inode-table block 3	Directory block 5
3	u in /home	Cold directory block 5	Inode 5
4	notes.txt in /home/u	Cold directory block 6; inode table cached	Inode 7
5	Open inode 7, permission succeeds	No further disk read	O1→I7, P1 fd4→O1, offset 0
6	P1 reads 8192 B	Data blocks 8,9	Data loaded, offset 8192
7	P2 independently opens same file	Inode/cache hit	New O2→I7, P2 fd9→O2, offset 0

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete path walk, open, and read states (continued)



The cold walk costs three metadata reads, inode table plus directories 5/6, and two data reads, totaling five. With everything cached it costs zero disk reads. Directory cache, inode cache, system-wide open-file table, and page cache are different layers. A hit in any layer proves no durability property.

The process fd table maps integers to descriptions and contains descriptor flags such as close-on-exec. An OFD is the kernel object created by open, containing offset, access/status flags, and an inode reference. An inode/FCB contains identity, size, permissions, and mapping and is associated with shared page-cache state. Independent opens do not merge offsets merely because the inode is shared.

Complete path walk, open, and read states (continued)

The textbook's teaching representation places some fields in per-process tables and has a system-wide FCB table. That abstraction must still identify the object owning a shared offset. This example uses the Linux/POSIX open/dup three-layer representation. Two descriptions now refer to I7; this is not one description with reference count two.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Independent-open and dup offset invariants

Continue with P1 fd4→O1, P2 fd9→O2, I7 size 12288. Reads succeed completely, without signals or other writers.

Event	Descriptor relation	O1 offset	O2 offset	Returned range
After opens	P1:4→O1; P2:9→O2	0	0	None
P1 read(fd4,8192)	Unchanged	8192	0	[0,8192)
P1 fd6=dup(fd4)	fd6 also points to O1	8192	0	No read
P1 read(fd6,4096)	fd4/fd6 share O1	12288	0	[8192,12288)
P2 lseek(fd9,4096, SEEK_SET)	Only O2 changes	12288	4096	Contents unchanged
P1 read(fd4,1)	O1 is at EOF	12288	4096	Zero bytes

For a successful r-byte read, $o' = o + r$, with $0 \le r \le \min(n, \max(0, size - o))$. Dup and fork inheritance of the same description share o; independent opens do not. Closing fd6 decreases O1's references while fd4 remains usable. Closing the last description does not remove a directory name. Unlink and final-open-reference release have different lifetimes.

Mount and VFS dispatch

Mount filesystem B's root inode 2 at A's /mnt. Resolving /mnt/log/x first finds A's mount point, crosses to (B, inode 2), then uses B's lookup for log and x. A and B may both have inode number 2, so identity requires filesystem/mount context as well as inode number.

VFS resolves dentry/inode and mount relationships, then dispatches a regular-file read through that filesystem's operations. A local filesystem maps blocks and submits block I/O; NFS forms RPCs. A common interface does not require every lower layer to use the same on-disk index block. Handle permissions, symlinks, .., and mounts during each resolution step. The five-read example explicitly excludes those extra traversals and permission-data I/Os.

Directory lookup costs

For equally likely queries over the n stored names and equal cost per name comparison, a linear list averages $(n+1)/2$ comparisons on successful lookup. If name i is queried with probability p_i, the mean is $\Sigma_i p_i i$. A hash table maps a name to a bucket and compares entries there. Fix

$$h(name) = \left(\sum_i code(char_i) \right) \bmod 8.$$

Bucket	Entries	Chain length
0	Empty	0
1	Empty	0
2	delta, epsilon, eta	3
3	gamma	1
4	beta, zeta	2
5	Empty	0
6	alpha	1
7	Empty	0

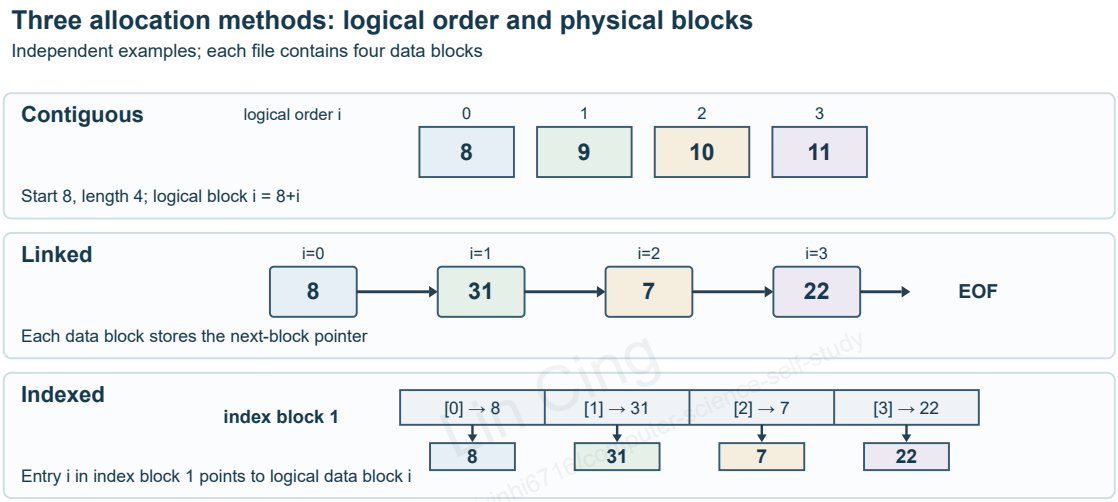
Directory lookup costs (continued)

Load is $\alpha = 7/8 = 0.875$. Uniform queries over these seven stored keys give $(1+1+1+1+2+2+3)/7 = 11/7 \approx 1.571$ comparisons. This averages query targets for a fixed function, not random functions.

Under an additional uniform pair-collision model over m buckets with fixed insertion order, insertion i has expected successful cost $1 + (i-1)/m$. Uniformly averaging stored keys gives $1 + (n-1)/(2m)$, or 1.375 for n=7, m=8. The common $1 + \alpha/2$ omits the finite-n correction. Observed 1.571 and expected 1.375 use different randomness and denominators. All keys colliding still yields O(n) worst case.

Eight chaining buckets can store more than eight names; a ninth name does not inherently overflow the table. Resize policy can depend on load and latency. Open addressing instead has at most one entry per slot. A visible intermediate rehash can omit an old entry or duplicate a name. Locks, versions, or a specified incremental migration protocol must preserve a coherent lookup result.

Three block allocation layouts



Direct, linked, and indexed access costs

Fix 4 KiB blocks and four-byte pointers, giving 1024 entries per pointer block. Retrieve zero-based logical block 2:

Method	Mapping	Disk reads	Main cost
Contiguous	Start11, length3	One data read	Fragmentation and relocation on growth
Linked	11→12→13	Three block reads	Follow links for random access
Indexed, index cached	Index entry→data13	One data read	Index space
Indexed, index cold	Index block, then data13	Two reads	Large-index memory/TLB costs

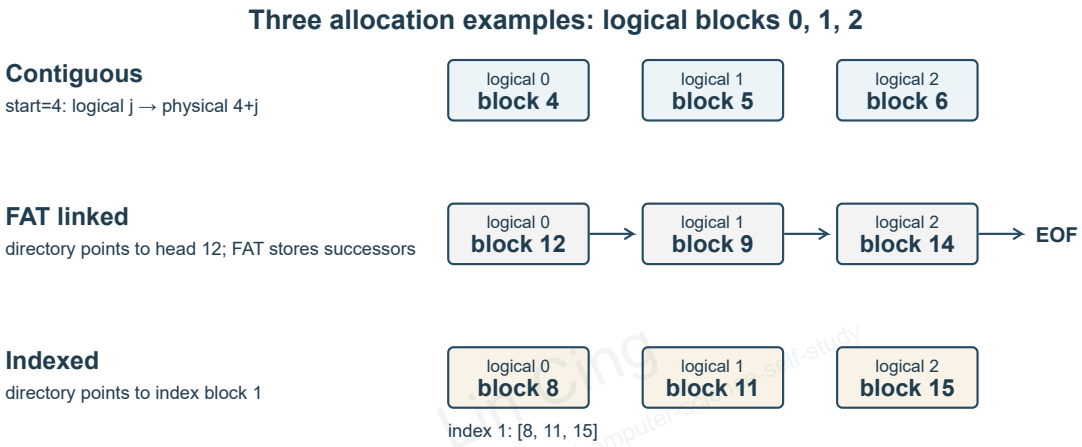
Contiguous mapping is start+i. Linked access starts at the head. Indexed access selects pointer i, with cost determined by its direct/single/double/triple level.

Direct, linked, and indexed access costs (continued)

Region	Logical blocks	Data capacity
Direct	0–11	48 KiB
Single indirect	12–1035	4 MiB
Double indirect	1036–1,049,611	4 GiB
Triple indirect	Following region	4 TiB

Offset 5 MiB exceeds the direct-plus-single region and requires double indirection. A sparse hole need not allocate a data block and reads as zero even though logical size covers it. A zero inode pointer is therefore not automatically corruption.

Three allocation methods: locate a file block



These are independent configurations. Numbers denote physical block IDs.

These are three independent examples, each with three data blocks numbered logically from zero. For contiguous allocation, `start=4, length=3` maps logical block 2 to physical block `4+2=6`. For FAT, the directory stores head 12; follow `12→9→14→EOF`, so logical block 2 is physical block 14. For indexed allocation, the directory points to index block 1, whose entries are 8, 11, 15; logical block 2 is physical block 15.

Two index levels: address calculation and I/O

Let data and index blocks both be 4 KiB, with 4-byte block pointers. Each index block holds `N=4096/4=1024` entries. Two levels can identify `N²=1,048,576` data blocks; logical block j, numbered from zero, uses `outer=floor(j/N)` and `inner=j mod N`.

For `j=1280`, outer=1 and inner=256. Read entry 1 of the first-level index to locate a second-level block, then its entry 256 to locate the data block. If the inode is already in memory, both index blocks and the target data block miss all relevant caches, each block requires one read, and other metadata and device caches are excluded, the model costs **2 index-block reads + 1 data-block read = 3 reads**. A cached first level saves one index read; a cached data block can save another. Three is a result of these assumptions, not a universal fixed cost.

Capacity of an inode index tree

Assume an inode has 12 direct pointers and one each of single, double, and triple indirect pointers. With 4 KiB blocks and 4-byte pointers, each index block contains `N=1024` pointers. A direct pointer identifies one data block; the three indirect forms identify up to `N`, `N²`, and `N³` data blocks, respectively.

Path	Addressable data blocks
12 direct	12
Single indirect	1,024
Double indirect	1,048,576
Triple indirect	1,073,741,824
Total	**1,074,791,436**

The data capacity is `1,074,791,436 × 4096 = 4,402,345,721,856 bytes`, about **4.00391 TiB**. This is the structural addressability limit of this index tree. It does not subtract physical storage for index metadata, nor does it establish that the filesystem's file-size field, block-address format, or available capacity permits such a file. Reading a hole within file size returns zero-valued data; a read beyond EOF returns 0 bytes instead.

Derive indirect indices from a byte offset

Let $q = \lfloor offset/4096 \rfloor$, $d = offset \bmod 4096$. Twelve logical blocks are direct, the next 1024 single-indirect, then 1024^2 double-indirect and 1024^3 triple-indirect. Use half-open ranges to avoid an extra endpoint.

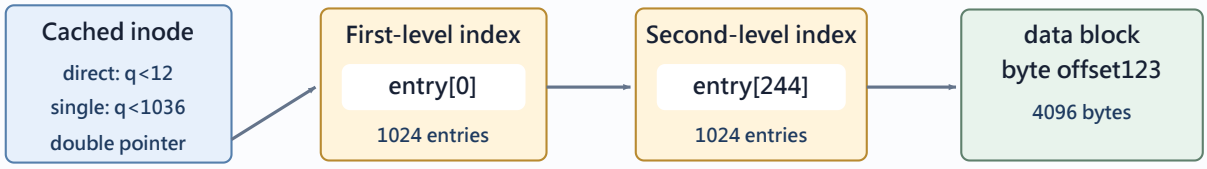
For offset=5 MiB+123=5,243,003 within the file size, q=1280, d=123. Subtract 12 direct and 1024 single blocks to obtain double-region index 244:

$$244 = 0 \cdot 1024 + 244.$$

Follow inode.double→first-level entry0→second-level entry244→data frame, then byte123. With the inode resident and both index levels/data cold, this costs two index reads plus one data read. A cached first index saves one metadata I/O.

Derive indirect indices from a byte offset (continued)

offset=5 MiB+123 → logical block1280 → double indirect [0][244]



244=1280−12−1024=0×1024+244. Cold access reads each yellow index, then the green data block.
Arrows are block pointers. Add 123 only after locating the final data block.

Total data addressability is $4096(12 + 1024 + 1024^2 + 1024^3)$ bytes, excluding index blocks from user data. Actual file limits also depend on size fields, block-number ranges, and filesystem rules. Within file size, a hole pointer can supply zeros without allocating a data block; returned zeros do not prove a zero-filled disk block exists. A regular read starting at EOF returns zero bytes, not an unbounded stream of zeros.

Allocation invariants must admit sparse holes and reflink/COW sharing. A hole has no data block. Reflink lets different inodes temporarily reference a COW-protected block, tracked by reference/version metadata and split on modification. Duplicate allocation means treating one block as independently writable without a sharing protocol. Removing one inode reference cannot free a block still used by a snapshot/reflink.

FAT versus pointers embedded in data blocks

For chain 11→12→13→EOF, embedding a four-byte next pointer in every 4096 B data block leaves 4092 B payload. Reaching the third chunk reads 11 and 12 for links, then 13, totaling three reads. Do not simultaneously count 4096 B payload and ignore the embedded pointer.

FAT stores FAT[11]=12, FAT[12]=13, FAT[13]=EOF in a separate table. If resident, locating the third block still needs two pointer lookups but only one data I/O. A cold FAT adds the table blocks actually read. FAT improves chain-metadata locality without changing CPU traversal from O(q) to O(1). Indexed allocation directly selects entry q.

An extent stores (logical_start, physical_start, length). Extent (0,8,6) maps logical 4 to physical12 by displacement. Occupied adjacent space can require another extent rather than moving the entire file, but extent-tree lookup and metadata costs remain separate.

Bitmap convention and a concrete allocation

Use 1=free,0=allocated, with block 0 first:

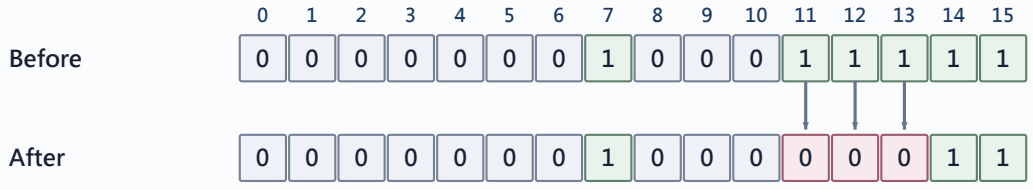
```
block:  0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
bits:   0 0 0 0 0 0 0 1 0 0  0  1  1  1  1  1
```

The string 0000000100011111 represents free set {7,11,12,13,14,15}. Inode7 already owns data8–10. Appending 12 KiB needs three blocks. A single contiguous extent starts at 11, because block 8 prevents a three-block run starting at 7.

```
after: 0000000100000011
free: {7,14,15}
```

Bitmap convention and a concrete allocation (continued)

First-fit allocates consecutive blocks 11, 12, 13: free → used



inode7: blocks [8,9,10] → [8,9,10,11,12,13]; size 12 KiB → 24 KiB

Green 1 = free; gray 0 = already allocated; red 0 = newly claimed. Free set = {7,14,15}.

Preserve that allocated blocks are not free and valid allocated inode pointers identify allocated blocks. Selecting volatile bits and updating disk bitmap/inode separately can crash into duplicate allocation or an orphan. A memory lock prevents concurrent races, not power-loss inconsistency.

$$\#bits = \#blocks, \qquad \#bytes = \left\lceil \frac{\#blocks}{8} \right\rceil .$$

A 1 TiB volume has 2^{28} four-KiB blocks, requiring 2^{25} bitmap bytes, or 32 MiB. Storage I/O still operates in blocks.

Four representations of the same free set

For {7,11,12,13,14,15}, a bitmap uses 16 bits. A free list can be head7→11→12→13→14→15→null. Allocating one block pops head7 and sets head=11; it need not scan the entire list. Finding a contiguous three-block run needs additional search/organization.

Grouping places several free addresses in a management block. With three addresses per group, the first two identify allocatable pages and the third links the next group. One read obtains two candidates. The format must define when the next-group management page itself becomes free.

Counting stores (start, length), here (7,1),(11,5). Allocating three from (11,5) returns [11,14), leaving (14,2). Freeing [11,14) merges it with (14,2) into (11,5). Free intervals are disjoint, adjacent intervals coalesced, and their union exactly the free set. A balanced offset tree supports O(log r) insertion/coalescing for r runs. Finding a sufficiently large run additionally needs a size index or subtree maximum; an offset tree alone does not make every query logarithmic.

The textbook ZFS space-map model logs ordered extent allocations/frees. Starting at (11,5), replay alloc(11,3) to obtain (14,2), then free(12,1) to obtain (12,1),(14,2). Rebuild from a checkpoint and remaining log instead of rewriting an entire bitmap in place. Preserve event order and transaction identity; replaying one allocation twice is not two new allocations.

Cache, read-ahead, and writeback

The cold example costs three metadata plus two data reads. Fully cached directory/inode/data state gives zero disk reads. Report hit rates by layer instead of replacing the model with a generic reduction percentage.

1. A cache hit avoids disk transfer but still costs lookup, copying, locking, and memory work.
2. Read-ahead loads consecutive blocks 8/9 to reduce seeking/latency; incorrect predictions perform extra reads.
3. Asynchronous writeback can return from write while data remains in dirty page cache. Flush/commit records determine the later durability point.

Fix 10,000 files, each exactly 33 KiB, without compression, deduplication, or holes. Four-KiB units allocate nine units, 36 KiB per file, wasting 3 KiB. Sixty-four-KiB units allocate one unit, wasting 31 KiB. The difference is 28 KiB per file, or 280,000 KiB=273.4375 MiB total. An average size alone cannot justify taking a ceiling after averaging because generally $E[\lceil S/B \rceil] \neq \lceil E[S]/B \rceil$.

In the simplified model of one four-byte mapping entry per unit, entries decrease from nine to one, saving only 32 bytes/file and 320,000 bytes total. These pointer bytes are separate from the 28 KiB data-space difference. Actual inode/extent/index metadata must follow its own structure.

Pointer sizes, unified cache, and synchronous writes

A four-byte pointer addresses 2^{32} block positions, theoretically $2^{32} \cdot 4\ \mathrm{KiB} = 16\ \mathrm{TiB}$. Actual file size also depends on indirection, block counts, and filesystem fields. Eight-byte pointers expand addressing but halve pointers per index block, so they do not automatically save space.

A unified page cache shares pages between mapped files and read/write. Consistency depends on PTEs, dirty tracking, writeback, and locking. Synchronous writing waits for its specified durability point. An asynchronous write may return in cache; its success alone does not prove survival of a later power loss.

Journal recovery at each failure point

Transaction T7 appends 12 KiB to a file initially sized 12 KiB. Blocks 11, 12, and 13 are exclusively reserved or locked from selection until commit or abort, so concurrent transactions cannot allocate them; old data blocks are not overwritten. After a crash, journal recovery completes before allocation or serving the new state resumes:

```
bitmap 11,12,13: free→used
I7.blocks += 11,12,13
I7.size = 24 KiB
data: blocks 11,12,13
```

Use a redo-only, no-steal ordered metadata journal. Descriptors, metadata after-images, and commit identify the transaction. A complete commit is atomically recognizable; torn/incomplete records do not count. Durable ordering is descriptor→data completion→metadata records→commit→checkpoint→transaction clearing. Before commit, uncommitted metadata cannot reach home locations. Device volatile caching must honor the required flush/barrier order.

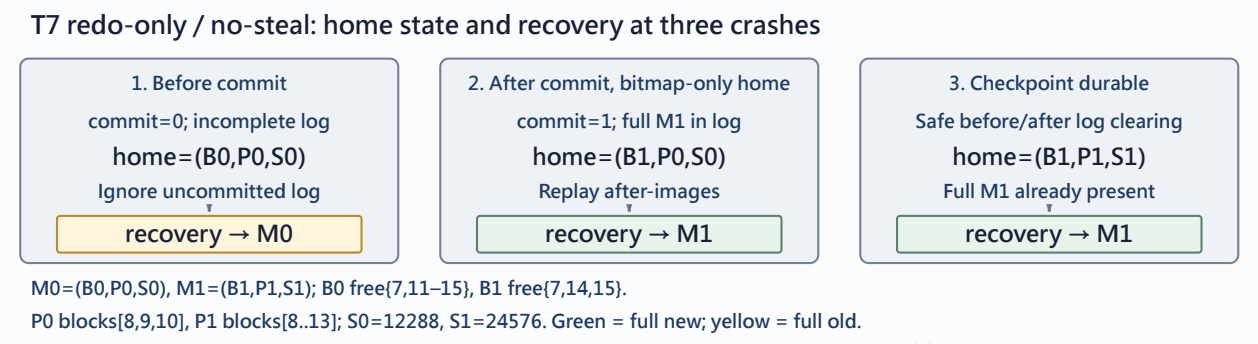
Journal recovery at each failure point (continued)

Crash point	Recovery evidence	Action	Visible state
Before descriptor	No T7	Discard	Old, size 12 KiB
After descriptor, before commit	Uncommitted	Ignore	Old
After commit, incomplete home	Committed	Idempotent redo	New, size 24 KiB
Home complete, log not cleared	Committed, possibly applied	Redo or recognize completion	Still new

Written data alone is not the commit criterion. This ordered model completes data before commit so metadata never publishes unwritten data. Metadata journaling alone does not give all user data the same transaction atomicity/durability.

Fsck scans directory/inode/bitmap relationships for orphan or duplicate blocks. It cannot reconstruct a user write that never persisted.

Journal recovery at each failure point (continued)



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Every legal crash prefix recovers to old or new

Let metadata $M = (bitmap, pointers, size)$. Old M_0 has pointers [8,9,10] and size 12288; new M_1 has [8,9,10,11,12,13] and size 24576.

Before commit, no-steal keeps home at M_0. New data may exist in blocks 11–13 but no published inode points there. Ignoring uncommitted records preserves M_0, and those free blocks can be allocated again.

After commit but before checkpoint completion, full after-images are already durable. They overwrite any partially updated home into M_1. If bitmap is new while inode pointers are old, recovery must redo all three fields before mounting. Redo assigns size←24576 and the entire new pointer array; repeating it still yields M_1, rather than appending again to 36 KiB.

Log clearing is allowed only after durable checkpoint, so a cleared transaction has full M_1 at home. These branches cover every legal crash prefix and prove atomic metadata visibility. Allowing precommit home changes would require undo or another protocol; discarding the uncommitted log would then be insufficient.

Journal, snapshot, and backup solve different failures

A journal repairs interrupted updates, not legitimate deletion followed by overwrite. A snapshot retains old reachable blocks but can fail with the same destroyed device. A backup stores a separate copy; recovery requires that copy to exist.

Let Sunday full backup be B0, Monday incremental B1 contain changes since Sunday, and Tuesday B2 changes since Monday. Recovering Tuesday after Wednesday failure requires B0→B1→B2, including deletion/rename records. B0+B2 omits files changed only Monday. Only a differential B2 containing all changes since B0 permits B0+B2. Untrusted mtimes or a changing live source require a consistent snapshot/point and reliable change records.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

NFSv3 handles, validation, and retries

Fix NFSv3, RFC1813, without importing NFSv4 OPEN/session state. Mount supplies a remote root handle. LOOKUP(directory_handle, name) obtains opaque child handles; READ carries handle, offset, count. NFSv3 has no remote OPEN/CLOSE procedure. Client fd/offset management is not a server POSIX OFD. A handle identifies an object, not an arbitrary resolvable filename.

Given /export handle H0, LOOKUP(H0, u)→H1, LOOKUP(H1, notes.txt)→H7, then READ(H7,4096,4096). Retrying READ adds no repeated mutation, but another client may have changed its bytes. Idempotent effects do not imply eternally identical results. If REMOVE's response is lost and another client recreates the same name, executing the old REMOVE as a new request can delete the wrong object.

Step	Client	Server	Evidence
1	Sends create X, id12	Creates X; reply lost	Request ID/effect/result
2	Times out, retries id12	Finds prior record	Returns prior result without another create

This requires a duplicate-request cache or the corresponding version's transaction/session mechanism, with explicit retention and crash behavior. Expiring finite caches do not establish permanent exactly-once execution. Client caches must validate relevant version/mtime/handle state; a cache hit is not a server commit.

UNSTABLE writes, COMMIT, and server restart

NFSv3 WRITE returns its actual stability level and a write verifier. UNSTABLE can leave data only in volatile memory. A client requiring durability sends COMMIT and compares its verifier with prior WRITE. A restart losing volatile data changes the verifier, requiring retransmission of unconfirmed data.

Event	Server state	Client obligation
WRITE offset 0, A returns UNSTABLE, v7	A may only be in RAM	Retain A and uncommitted range
Restart changes verifier to v8	A may be lost	Prior WRITE success proves no durability
COMMIT returns v8 instead of v7	A's persistence unproven	Retransmit A, then request stability
Retransmitted write completes stably with matching verifier	Protocol-stable storage reached	Release retransmission obligation

Attribute/data caches can still be stale. Close-to-open and timeout validation are client policies, not global linearizability implied by the NFSv3 name. A remote mount does not automatically import the server's other mounts; clients resolve their own mount tables.

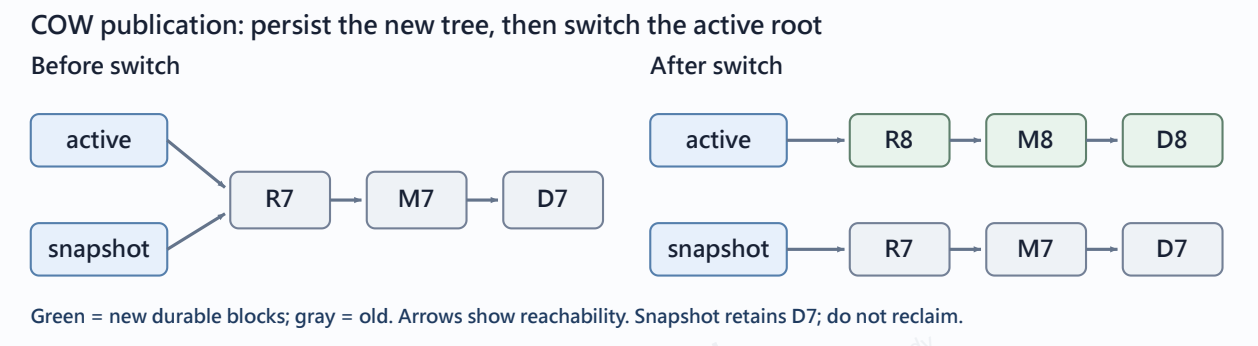
COW root publication and snapshot sharing

Version7 has R7→M7→D7, reachable from both active and snapshot roots. Counting two retained roots is a reachability model, not a claim that WAFL stores integer refcount2 in every block. The textbook's snapshot bitmap records snapshot usage.

Stage	Active root	New blocks	Old-tree reachability
v7	R7	None	Active/snapshot reach R7/M7/D7
Prepare v8	R7	D8, M8, R8	Transaction reserves unpublished R8; old tree retained
Durable	R7	D8/M8/R8 durable	Crash before switch still selects v7
Atomic switch	R8	Published	Active reaches new tree; snapshot reaches old

Write new data and all new metadata before recoverable atomic root publication. This does not assume arbitrary-size hardware root writes are intrinsically atomic. A snapshot retaining R7 prevents reclaiming D7. Only removal of all retaining roots permits reclamation. Transactions also reserve unpublished new blocks so other allocators cannot reuse them merely because no public root reaches them yet.

COW root publication and snapshot sharing (continued)



One data-block update can require new data, metadata, and root blocks. Reclamation must account for active and snapshot references instead of freeing a block merely because it is old.

Four mappings from names to blocks

Mapping	Invariant	Failure
Path/directory→inode	Namespace reaches intended object	Textual symlink-suffix cancellation
fd→OFD→inode	Correct offset/reference ownership	Treat dup offsets as independent
Logical→physical block or hole	Mapping agrees with allocation	Treat hole as disk block 0
Pointers/size↔free space	No incompatible reallocation	Mark free while old pointer remains usable

Memory locks coordinate concurrent updates. Crash consistency of several metadata fields additionally requires journaling/COW. Valid, dirty, writeback, and device-completed cache states are not one written state.

Journal and COW publication points

T7 uses redo-only/no-steal; complete after-images can be replayed after durable commit.

Crash prefix	Recovery
Commit not durable	Home remains old; no uncommitted home writes
Commit durable, home partial	Replay every after-image into consistent new
Home/checkpoint durable	Reclaim log only under the proved truncation rule

After-image assignments are idempotent. Precommit home writes require undo or another mechanism. COW instead persists new data/metadata before recoverable root publication; retained snapshots forbid premature reclamation.

Metadata consistency alone is not a data transaction, backup, or cross-client coherence.

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 12.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study